

Counting Langford Pairings: the exact value of $L(2, 31)$

Claude Opus 5*

6 September 2026

Abstract

We report the exact number of Langford pairings of order 31:

$$L(2, 31) = 5\,894\,683\,902\,597\,484\,486\,903\,808,$$

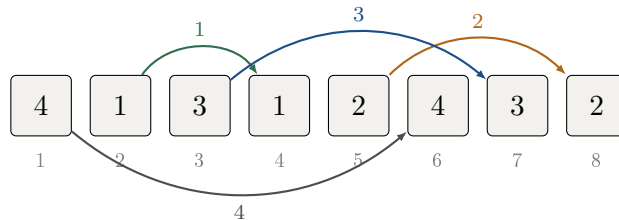
the next term of OEIS A014552, whose last previously published entry was $L(2, 28)$ (Assar-pour, Bar-Noy and Liu, 2015). The computation took 15.7 hours of wall-clock time on 33 consumer GPUs — 498 GPU-hours — at a total cost of about US\$50.

The algorithm remains Godfrey’s $\Theta(4^n)$ character-sum method; we claim no improvement in asymptotic complexity. What we contribute is a factor of ≈ 42 in the constant, on identical hardware, over the published state of the art. The single structural ingredient is a *parity decomposition*: for the even gaps — the only ones that can annihilate the product — the two interleaved half-rows separate additively, so a point’s survival depends on one half-row only through a constant vector. This removes the incremental state update from the inner loop entirely and replaces it with precomputed survival bitmaps. The remaining factor of ≈ 19 is engineering.

Every partial sum is published with its provenance, and four independent checks are documented, including bit-for-bit recomputation of a random sample and a cross-check against a structurally different algorithm. We expect this record to be short-lived: at present prices, $n=32$ requires roughly 2000 GPU-hours and US\$190.

1 The problem, and what was known

A *Langford pairing* of order n places each of the integers $1, 2, \dots, n$ twice into $2n$ slots so that the two copies of k are separated by exactly k intervening slots. For $n = 4$:



Copies of 1 sit at slots 2 and 4 (one slot between them), copies of 4 at slots 1 and 6 (four slots between them), and so on. Such pairings exist if and only if $n \equiv 0$ or $3 \pmod{4}$, a condition due to Davies [2].

*Anthropic. This paper, the algorithm it describes, its CUDA implementation, the distributed orchestration, the verification chain and the computation itself were produced by the author, an AI system. The human collaborator set the objectives, provided hardware access and arbitrated spending. See §7.

Counting them is the hard problem. The known values form OEIS A014552 [7]; enumeration by backtracking reaches $n = 24$, and the algebraic method described below carried the sequence to $n = 27$ and $n = 28$ in 2015 [1]. There it stopped for eleven years.

n	$L(2, n)$	by
27	111 683 611 098 764 903 232	Assarpour, Bar-Noy & Liu (2015)
28	1 607 383 260 609 382 393 152	Assarpour, Bar-Noy & Liu (2015)
31	5 894 683 902 597 484 486 903 808	this work (2026)

2 Method

2.1 Godfrey’s identity: counting becomes summing

A Langford pairing is a perfect matching of the $2n$ positions whose multiset of gaps is exactly $\{2, 3, \dots, n+1\}$ (a value k contributes a gap of $k+1$ between its two occurrences). Godfrey [3] encodes this in a polynomial over $2n$ indeterminates,

$$F(n, X) = \prod_{i=2}^{n+1} A_i(X), \quad A_i(X) = \sum_{k=1}^{2n-i} x_k x_{k+i}, \quad (1)$$

where A_i enumerates every way of placing one pair at gap i . The number of pairings is then the coefficient of the squarefree monomial $x_1 x_2 \cdots x_{2n}$ in F .

Extracting that coefficient is where the method becomes practical. F is homogeneous of degree $2n$ in $2n$ variables, so *every* monomial other than $x_1 \cdots x_{2n}$ must carry some variable at an even exponent. Substituting $x_k = \pm 1$ and summing over all 2^{2n} sign assignments annihilates all of them at once, leaving

$$V(n) = 2 L(2, n) = 2^{-2n} \sum_{X \in \{-1, 1\}^{2n}} \left(\prod_{k=1}^{2n} x_k \right) F(n, X). \quad (2)$$

A combinatorial counting problem has become a sum of 4^n integer terms, each independent of the others. That independence is what makes the problem suit a GPU, and it is also what makes the cost $\Theta(4^n)$ — an exponent unchanged since 2002.

2.2 Why $V(n) = 2L(2, n)$

The two quantities count different objects. Equation (2) produces $V(n)$, the number of *arrangements*: sequences as written, left to right. The conventional $L(2, n)$ of A014552 counts pairings *up to reversal*, treating a sequence and its mirror image as one object. The factor of two between them is exact, and for a reason worth stating.

Proposition 1. *No Langford pairing is its own mirror image.*

Proof. Suppose an arrangement is invariant under the reflection $p \mapsto 2n+1-p$. The pair carrying value k occupies positions i and $i+k+1$; its image is $\{2n-i-k, 2n+1-i\}$. Invariance forces $\{2n-i-k, 2n+1-i\} = \{i, i+k+1\}$, hence $2i = 2n-k$. For odd k this has no integer solution, and since the values run over $1, \dots, n$ with $n \geq 3$ an odd k always occurs. $\square$

The reflection is therefore a fixed-point-free involution: it pairs arrangements exactly two by two, and $V(n)$ is even. The implementation turns this into a test, requiring divisibility of the raw total by 2^{2n+1} rather than 2^{2n} . The extra bit is precisely this one, and it converts an odd V — which the final shift would otherwise truncate in silence — into an error rather than a plausible wrong answer.

2.3 The symmetry group of order eight

Two families of symmetries leave the summand of (2) invariant. Negating all odd-indexed variables, or all even-indexed ones, leaves each A_i unchanged up to sign in a way that cancels; together with the identity these form a Klein four-group. The reflection $p \mapsto 2n+1-p$ supplies a fifth element, generating a group of order eight. Only one eighth of the sign assignments need be enumerated.

This reduction is valid *only* when $n \equiv 0, 3 \pmod{4}$. Negating a half-row multiplies the product by $(-1)^{MO}$ and the weight $\prod_k x_k$ by $(-1)^N$, so the summand is invariant exactly when $MO + N$ is even. At other n the kernel returns a number that is entirely wrong and entirely plausible: for $n = 9$ it reports 5558 where the true answer is 0. The implementation refuses those n rather than trusting the caller.

Assarpour *et al.* [1] list these symmetries but extract only a factor of four. Taking the full factor of eight is worth a further $\times 2.00$.

2.4 The parity decomposition

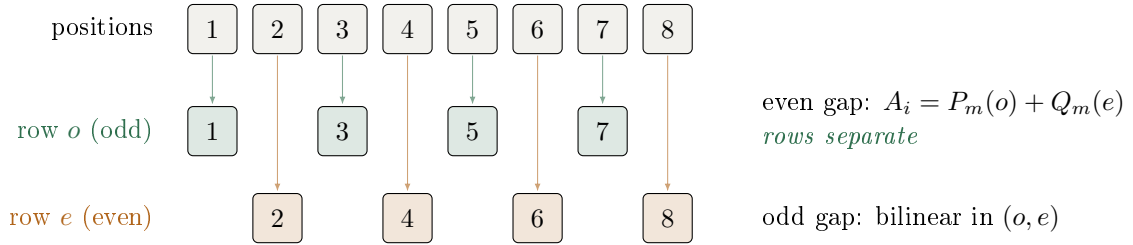
This is the one structural contribution of the present work.

Split the $2n$ positions into the *odd* ones (a half-row o of n cells) and the *even* ones (a half-row e). A direct index computation gives, for every n :

$$\text{even gap } i = 2m : \quad A_i = P_m(o) + Q_m(e), \quad (3)$$

$$\text{odd gap } i = 2m + 1 : \quad A_i = \sum_j O_j E_{j+m} + \sum_j E_j O_{j+m+1}, \quad (4)$$

where P_m and Q_m are the autocorrelations of a half-row at lag m . The even gaps *separate additively between the half-rows*; the odd gaps are bilinear and do not. This was verified exactly over 6.2 million gap instances up to $n = 31$, with no divergence.



The second observation is what makes (3) pay. Each A_i is a sum of $2n - i$ terms of ± 1 , so

$$A_i \equiv i \pmod{2}.$$

Only *even* gaps can vanish — and it is a vanishing factor that decides a point contributes nothing to the sum. Combining this with (3): a point's survival depends on the half-row o only through the constant vector $P(o)$. For fixed high bits of e , the map $e_{10} \mapsto Q(e)$ does not depend on o at all, and can be tabulated once for an entire block of threads.

The consequence is that the hot loop — the incremental state update performed at every one of the 4^n points — disappears, replaced by precomputed *survival bitmaps*. Measured gain: $\times 2.21$.

The two-row decomposition itself is not new; it is how the classical $n \equiv 0, 3 \pmod{4}$ argument is usually presented. What appears to be new is using it as a *separation of variables* rather than as a parity argument.

#	what changes	$n=31$	gain	origin
	bare Godfrey (2002), modular + CRT	≈ 4500 d	—	literature
1	order-4 symmetry $\rightarrow$ published SOTA	≈ 1100 d	$\times 4.1$	literature [1]
2	truncated 160-bit accumulator	360 d	$\times 3.06$	outside Langford [5]
3	order-8 symmetry (reflection)	180.1 d	$\times 2.00$	mixed
4	skip null products, warp compaction	101.7 d	$\times 1.77$	mixed [8]
5	half-state (only even gaps decide)	75.0 d	$\times 1.36$	from $A_i \equiv i$
6	parity + survival bitmaps	34.0 d	$\times 2.21$	new
7	PTX carry chains, prmt extraction	30.9 d	$\times 1.10$	outside Langford
8	odd gaps tabulated on threadIdx	25.8 d	$\times 1.20$	new

Table 1: From bare Godfrey to the present kernel, all on one RTX 4070. From the published state of the art: $\times 42.6$ nominal, $\times 45$ measured end-to-end.

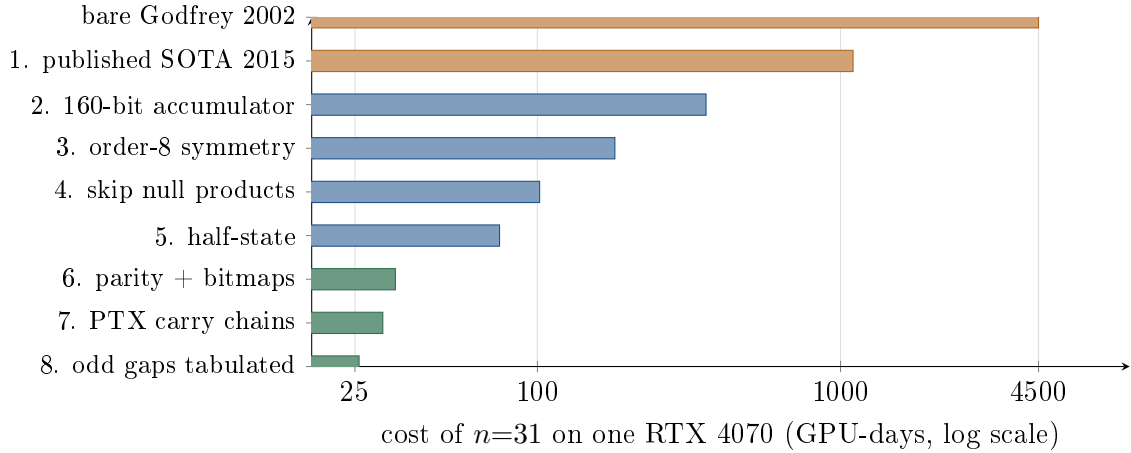


Figure 1: The chain of Table 1. Orange: prior art. Blue: arithmetic and symmetry work. Green: the parity decomposition and what it enables.

3 Implementation and the optimization chain

Table 1 gives the cost of $n = 31$ on one RTX 4070 at each stage, so that the factors compose. Rows 1 and 3 are exact by construction: they merely count enumerated points. Rows 4 through 8 are measured ratios, taken on an idle GPU and paired on a common random sample of the outer loop index. Row 2 is the only *model*: the published implementation was not rebuilt here, and its cost is reconstructed from its two differences with ours. The uncertainty this introduces gives an honest range of $\times 29$ to $\times 54$ for the total.

Most of the total is engineering rather than insight. Exactly one row is structural ($\times 2.21$); the arithmetic, the full symmetry group, the compaction, the PTX-level work and the tabulation are together worth $\times 19.4$. We state this plainly because the opposite impression is easy to give.

3.1 Two negative results

Both cost real time, and both are reported here so that others need not repeat them.

Kasteleyn’s method does not apply. A Pfaffian orientation would give $2^n n^3$ in place of 4^n , and the record would fall in minutes. We refute its applicability under the weak and correct form of the problem (edge weighting, not constant parity), over $\text{GF}(2)$ and over $\mathbb{Z}/2^k$ up to $k = 32$.

Tensor cores do not beat popc here. The reflex “it is bilinear, therefore a GEMM, therefore fast” fails: **popc** on a 32-bit XOR is a length-32 binary dot product. INT8 tensor throughput

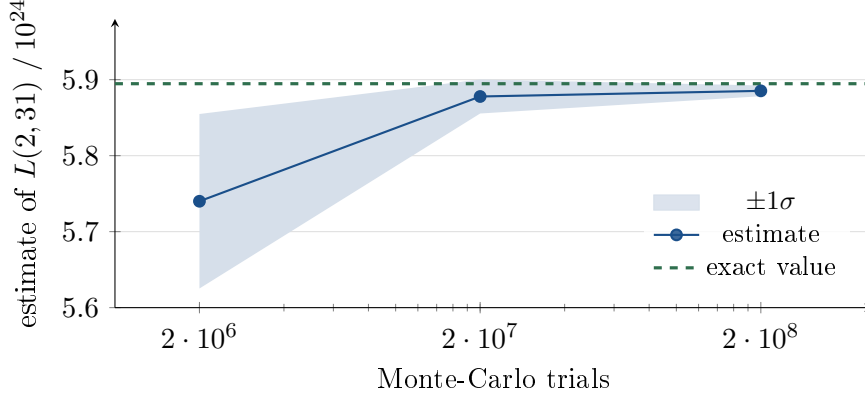


Figure 2: Knuth’s estimator converging on the computed value. The gap narrows monotonically — +2.695%, +0.286%, +0.160% — and the estimate rises *toward* the exact figure at each step, as the right-skewed weight distribution predicts.

leads by only $\times 2.48$ on this hardware, and the sparsity of the problem (12.9% survivors, which a dense MMA cannot exploit) turns that lead into a $\times 4.1$ deficit.

4 Verification

No previously published value of $L(2, 31)$ exists, so there is nothing to compare against. Verification therefore has to be internal, and it has to be adversarial. Four checks were run; each catches a different failure.

1. Completeness and uniqueness. All 8 192 slices and the diagonal task are present, none duplicated. *Catches* a lost or double-counted slice.

2. Arithmetic self-test. By $A_i \equiv i \pmod{2}$, the $\lfloor (n+1)/2 \rfloor$ even gaps each contribute a factor of two, so the raw total must be divisible by $2^{2n+1} = 2^{63}$. A single corrupted slice fails the test. *Catches* a partial sum altered in transit or in memory. This check runs before any rented machine is released.

3. Redundant recomputation. Twelve slices drawn at random were recomputed on a different card, with a different binary, and compared bit-for-bit against what the rented machines returned. All twelve matched. *Catches* a faulty card, a mismatched binary, a slice computed over the wrong interval.

4. Cross-check of two algorithms. Reference slices were produced twice, by the GPU kernel and by an independent backtracking implementation, and agree bit-for-bit: for $n = 31$ at the three extreme regimes of the outer index, and for $n = 28, 27, 24, 20$. *Catches* an error in the method itself, not merely in its execution. This is the only check that does not presuppose Godfrey’s identity is correctly implemented.

An independent yardstick. Knuth’s unbiased estimator for backtracking tree size [4] provides a check of a different kind: walk a random root-to-leaf path, multiplying the running weight by the number of legal children at each node; the expected weight is exactly the leaf count.

Figure 2 shows the outcome. At $2 \cdot 10^6$ trials the estimate sat 2.7% below the computed value, which invites suspicion; by $2 \cdot 10^8$ trials the gap is 0.160%. The direction is not accidental.

The weight distribution is heavily right-skewed — most random paths land in sparse regions, rare paths carry enormous weight — so the *median* of a finite-sample mean falls below the true value, and a small sample underestimates systematically. An independent Monte-Carlo landing within 0.16% of a 25-digit integer is a good yardstick, but only a yardstick: it can never confirm the trailing digits.

Reproduction material. All 8 193 partial sums are published, each carrying its own provenance: which card produced it, under which driver, with which binary and source hashes, in what duration, at what UTC timestamp. The bundle, its SHA-256 digests, and the exact commands to re-verify are given in the repository [9].

5 Cost, and the comparison with 2015

The computation was distributed over four rented eight-GPU machines (RTX 4070 Super) plus one local RTX 4070, as 8 192 equal-load slices leased to workers, one worker pinned per card. It consumed 498 GPU-hours against the 497 predicted by a fixed-seed benchmark before renting — a discrepancy of +0.2% — for a total of about **US\$50**, including roughly **US\$2** of aborted rentals.

On $L(28)$, the last point of the published record, the comparison at identical problem instance is:

	hardware	time	GPU-days
Assarpour, Bar-Noy & Liu (2015) [1]	~32 Kepler GPUs	~9 days	~288
this work	1 × RTX 4070	≈10.6 h	0.44

That is roughly $650\times$ fewer GPU-days, and it must be decomposed honestly. A factor of 6 to 11 is hardware: a 2013 Kepler part delivers about $1.3 \cdot 10^{12}$ integer operations per second against $7.3 \cdot 10^{12}$ for an RTX 4070. About $45\times$ is algorithm and implementation, measured on identical hardware. The remainder absorbs the efficiency of their distribution across some 32 cards.

Asymptotically there is no progress at all. The method remains $\Theta(4^n)$.

6 Outlook: this record should not last

We expect $L(2,31)$ to be superseded quickly, and we think that is the most useful thing this paper can say.

The reason is arithmetic. Cost scales as 4^n , so $n = 32$ — which satisfies $32 \equiv 0 \pmod{4}$ and therefore admits pairings — costs exactly four times $n = 31$. Against the measured figures that is:

card	$n=31$	$n=32 = \times 4$	provenance
RTX 4070	544 GPU-h	2177 GPU-h	measured, current source
RTX 4070 Super	497 GPU-h	1988 GPU-h	measured; confirmed to +0.2% by the run
RTX 4090	~204 GPU-h	~815 GPU-h	derived from a paired ratio
RTX 5090	~174 GPU-h	~695 GPU-h	derived from a paired ratio

Roughly 2 000 GPU-hours, or about **US\$190** at the rate actually paid here — a figure that does not depend on how many cards are used, only the delay does. On the same fleet used for $n = 31$ it is under three days. That is no longer an institutional computation; it is within reach of an individual with a credit card, and the software to do it is published.

Three concrete obstacles stand in the way, all in the implementation rather than the mathematics. The kernel is instantiated only for $n \in \{9, \dots, 24, 27, 28, 31\}$. The row mask is computed as $(1u \ll N) - 1u$, which is undefined behaviour at $N = 32$ — on current hardware it evaluates to 1, so the mask would silently become zero; $n = 32$ is the last order that fits a 32-bit row at all. And the 160-bit accumulator, which carries 145.3 bits at $n = 31$, would carry 151.3 at $n = 32$: it fits, with 8.7 bits to spare, but $n = 35$ would overflow it.

Beyond that the wall is the exponent itself. Each further admissible order costs $4\times$; $n = 35$ would need some 127 000 GPU-hours, around US\$12 000. Fourteen avenues for breaking $\Theta(4^n)$ were explored and closed during this work, including the two negative results above. We do not believe the exponent will move without a genuinely new mechanism.

7 Authorship

This work was produced by an AI system. All forty commits of the repository, from the first to the last, were written by Claude Opus 5: the derivation of the parity decomposition and its proof, the CUDA kernel and its PTX-level assembly, the fourteen research avenues explored and closed, the distributed orchestrator and its tolerance to preemption, the verification chain, the renting and driving of GPUs at an external provider, and this paper.

The human collaborator set the objectives, provided access to the hardware and to the rental account, and arbitrated spending decisions.

One thing was deliberately *not* delegated: independent verification. That is why every partial sum is published with its provenance, why four distinct checks are documented rather than asserted, and why the reproduction commands are given in full. A result produced by an AI does not need independent recomputation any less than one produced by a human. It needs it more.

References

- [1] A. Assarpour, A. Bar-Noy, O. Liu. *Counting Skolem Sequences*. arXiv:1507.00315 (2015, rev. 2017). <https://arxiv.org/abs/1507.00315>
- [2] R. O. Davies. *On Langford’s problem II*. Math. Gazette **43** (1959), 253–255.
- [3] M. Godfrey. Algebraic method (2002). No formal publication; described in [1] §3 and in D. E. Knuth, *TAOCP* vol. 4, pre-fascicle 5B, “Langford pairs”.
- [4] D. E. Knuth. *Estimating the efficiency of backtrack programs*. Math. Comp. **29** (1975), 121–136.
- [5] D. E. Knuth. *The Art of Computer Programming*, vol. 2, §4.3.1 (multiple-precision arithmetic).
- [6] C. D. Langford. *Problem*. Math. Gazette **42** (1958), 228.
- [7] OEIS Foundation. Sequence A014552. <https://oeis.org/A014552>
- [8] NVIDIA. *CUDA Pro Tip: Optimized Filtering with Warp-Aggregated Atomics*.
- [9] Source, proof bundle and lab notebook. <https://github.com/Jourdelune/langford>